arXiv:2601.03315v1 [cs.LG] 6 Jan 2026

Why LLMs Aren't Scientists Yet: Lessons from Four Autonomous Research Attempts

Dhruv Trehan Paras Chopra

{dhruv.trehan, paras}@lossfunk.com

January 2026

Abstract

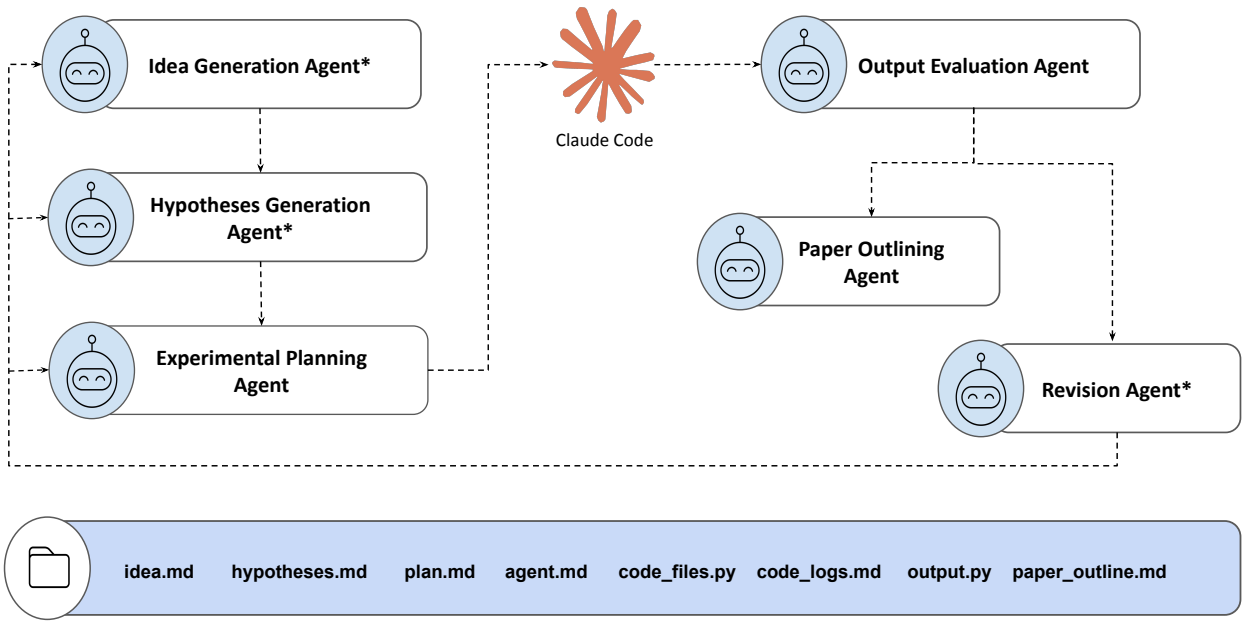
We report a case study of four end-to-end attempts to autonomously generate ML research papers using a pipeline of six LLM agents mapped to stages of the scientific workflow. Of these four, three attempts failed during implementation or evaluation. One completed the pipeline and was accepted to Agents4Science 2025, an experimental inaugural venue that required AI systems as first authors, passing both human and multi-AI review. From these attempts, we document six recurring failure modes: bias toward training data defaults, implementation drift under execution pressure, memory and context degradation across long-horizon tasks, overexcitement that declares success despite obvious failures, insufficient domain intelligence, and weak scientific taste in experimental design. We conclude by discussing four design principles for more robust AI-scientist systems, implications for autonomous scientific discovery, and we release all prompts, artifacts, and outputs at `https://github.com/Lossfunk/ai-scientist-artefacts-v1`.

1 Problem Definition and System Overview

The question we set out with was: could state-of-the-art reasoning LLMs go from a research idea to a research paper with a high degree of autonomy, minimal code scaffolding, and the most basic tools? Various AI Scientist systems proposed in research papers already relied on a high

degree of domain-specific pre-definition of the workflow or framing the problem statement in a system-specific way. Tree-search systems like Sakana's [1] would have required complex meta-orchestration, contradicting our minimal scaffolding goal. Similarly, Google's AlphaEvolve system [2] requires a clear verification metric defined by a human expert in advance.

We were interested in exploring how far current LLMs can go without significant scaffolding or inputs from humans. In line with this, we limited our scope to computational sciences, and specifically Machine Learning, since experiments in this domain could be done completely digitally. Figure 1 shows a high-level diagram of our system design.



* *Used an external review for iterative refinement.*

Figure 1: Autonomous Research Pipeline: High-level diagram showing the interaction between the six agent modules and the shared file system artifacts (`idea.md` to `paper_outline.md`) used to maintain context.

Our final system consisted of the following six distinct modules, all using Gemini 2.5 Pro primarily because of its long context length. Prompts and output formats for each of these are available in our GitHub repository[1].

1. **Idea Generation Agent**, which generated research ideas by combining insights from two input papers in a subdomain and outputted an idea in a structured format. More details on the idea generation process are included in Section 2.

2. **Hypotheses Generation Agent**, which took the generated research idea and proposed testable and falsifiable hypotheses with clear claim statements, datasets, baselines, and metrics to observe to guide experimental planning in subsequent stages.

[1] `https://github.com/Lossfunk/ai-scientist-artefacts-v1`

3. **Experiments Planning Agent**, which converts hypotheses and ideas into detailed implementation plans (`plan.md` and `agent.md` files) containing step-by-step tasks, project structure, method clarifications, and failure mode controls for autonomous execution by Claude Code on Modal infrastructure.

4. **Experimental Output Evaluation Agent**, which employed a two-tiered evaluation approach, primarily reviewing the output of an experiment for hypothesis implementation fidelity and statistical validity. The same agent was later used to conduct a paper readiness check for a completed hypotheses suite to determine if sufficient insight had emerged to proceed to paper writing tasks.

5. **Revision Agent**, which automatically determined the next step in the research process when experimental output evaluation indicated failure, choosing between revising the idea, revising the hypotheses suite, or requesting a LLM-mentor feedback. The agent was triggered infrequently since most failed experiments were terminated by human decision rather than routed through the revision process.

6. **Paper Outlining Agent**, which reviewed the complete experimental output, as well as all relevant context docs across stages, to outline a research paper including descriptions of visualizations required. Claude Code then completed this outline, section by section, into a full paper.

Experimental Implementation and Paper Writing - The experimental implementation and paper writing were executed by Claude Code using Claude Opus 4.1 and Claude Sonnet 4 for iterative code development, execution on Modal infrastructure, and manuscript development from outline to full text.

For paper-writing, we adopted a three-stage, iterative approach: generating a detailed outline, completing sections sequentially, and conducting two rounds of human-Claude Code collaborative editing—first to ensure continuity and flow, then to temper overoptimistic claims about results. In our experience, human intervention remained necessary for quality control during the paper-writing phase because initial autonomous drafts generated with prompting were technically accurate but tended toward overly mechanical prose or lacked narrative nuance.

Tools and Model Access - The model was accessed on Google AI Studio using an agentic prompt, including repository location and the following four tool definitions:

- `read_file` - This enabled the LLM to read context files including previously generated documents, mentor notes from external LLM review, or outputs from code implementation and execution.
- `write_file` - This enabled the LLM to save context files.